

TD7: Ingeniería de Datos

Integración de Datos

Integración de Datos

— Del dato crudo al insight:
arquitectura de un Data
Warehouse moderno. —

Por qué un Data Warehouse?

El problema que resuelve

Tu app tiene una DB transaccional. ¿Por qué no consultar ahí?

- Está optimizada para escribir, no para queries analíticas.
- Reporting pesado puede tirar abajo la app de producción.
- Cambia con cada deploy → difícil mantener historia.
- Tenés N fuentes (DB, CRM, eventos, planillas...) y nadie las cruza.
- Áreas no técnicas necesitan SQL "simple", no joins de 12 tablas.

Qué es un Data Warehouse?

Definición práctica

Una base diseñada para ANALIZAR, no para operar.

DB transaccional (OLTP)

- Optimizada para INSERT/UPDATE
- Datos normalizados
- Estado actual
- Latencia baja por transacción

Data Warehouse (OLAP)

- Optimizada para LEER mucho a la vez
- Datos desnormalizados (star schema)
- Historia + estado actual
- Latencia ok, throughput alto

Qué es Medallion?

Arquitectura en capas para procesar datos

Tres capas, cada una con un propósito claro.

Bronze	Silver	Gold
Raw / "tal cual" • Copia fiel de la fuente • Append-only, con timestamp • Auditoría: ¿qué llegó y cuándo?	Datos limpios y conformados • Tipificación y standarización de datos • Eliminación de duplicados • Integración de datos entre fuentes • Aplicación de reglas de negocio • Una única "verdad" por entidad	Listo para análisis • Modelado en star schema • Agregaciones y métricas • Lo que ve Metabase / BI

Por qué separar en capas?

Qué ganamos con esta arquitectura

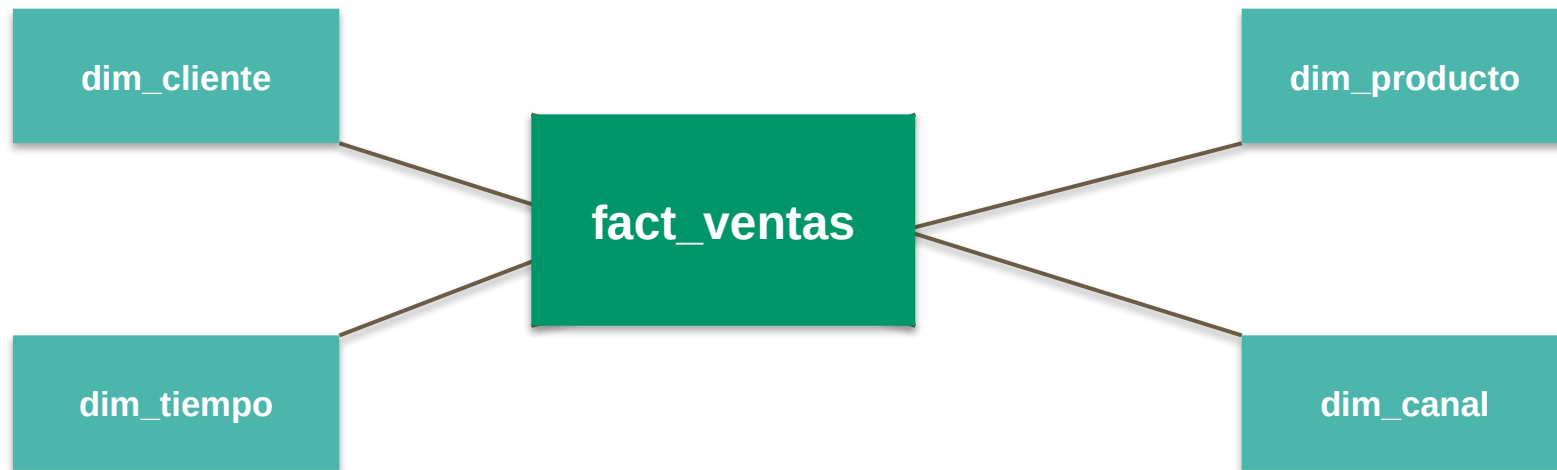
No es burocracia, son superpoderes:

- Reprocesar es fácil: tenés bronze, podés recalcular silver y gold cuando quieras.
- Si un dato sale mal, sabés EN QUÉ CAPA se rompió.
- Cada capa tiene su test → la calidad se chequea en el lugar correcto.
- Idempotencia: correr el job 10 veces deja la tabla igual que correrlo 1 vez.
- Equipos diferentes pueden trabajar en capas distintas sin pisarse.

Star Schema

Cómo modelamos la capa Gold

Una tabla de hechos (fact) en el centro, dimensiones alrededor.



Optimizado para queries: pocos joins, columnas anchas.

Fact vs Dimension

Qué guarda cada tipo de tabla

Fact tables

- Eventos que pasaron (ventas, clicks, envíos)
- Muchas filas, muy angostas
- Foreign keys a las dimensiones
- Métricas: cantidad, monto, duración
- Tienen una fecha asociada

Dimension tables

- Contexto: quién, qué, dónde, cuándo
- Pocas filas, muchas columnas
- Atributos descriptivos (nombre, categoría)
- Pueden cambiar en el tiempo → SCD
- Una surrogate key + business key

Slowly Changing Dimensions

Qué pasa cuando un atributo de una dimensión cambia?

Ejemplo: un cliente se muda de Buenos Aires a Madrid.

Tipo 1: pisar

- Update directo sobre la fila
- No queda rastro del valor anterior
- Reportes históricos cambian
- Simple, barato

Tipo 2: versionar

- Nueva fila con valid_from / valid_to
- Una sola activa por business key
- Reportes históricos se mantienen
- Tablas más grandes, joins por fecha

Data Quality

Qué chequeamos en cada capa

Si nadie testea los datos, los reportes mienten en silencio.

Unicidad

Las PK son únicas (no hay dos filas iguales)

No nulos

Las columnas críticas no tienen nulls inesperados

Rangos

Los precios son positivos, las fechas no son del año 3000

Referencias

Toda FK existe en su dimensión

Frescura

La tabla se actualizó en las últimas N horas

Data Governance

Que alguien encuentre el dato y confíe en él

Cuatro preguntas que tiene que poder responder cualquier usuario:

- ¿Qué tablas y columnas existen? → catálogo de datos
- ¿De dónde viene este número? → lineage (de qué fuente y por qué transformaciones pasó)
- ¿Cuándo se actualizó por última vez? → freshness
- ¿Quién es el dueño? → ownership (a quién pregunto si algo está mal)

Orquestación

Quién dispara qué, cuándo y en qué orden



Pasamos de scripts sueltos en cron a un **DAG con dependencias**.



Dependencias

Cada tarea espera a que sus dependencias terminen.
Ejemplo: gold no corre si silver no terminó.



Retries automáticos

Si algo falla, se reintenta automáticamente según la configuración.



Idempotencia

Correr dos veces el mismo job no rompe datos.
El resultado es siempre el mismo.



Particionado por fecha

Podés reprocesar solo lo que necesitás.
Ejemplo: 2026-03-12.



Backfills

Cargar histórico desde cero sin reescribir código.
Ideal para reconstrucciones o nuevos rangos de fechas.

Ejemplo de
flujo



Bronze
Ingesta



Silver
Limpieza y
conformado



Gold
Datos listos para
consumo

El stack

Lo que vamos a levantar con Docker

Cuatro piezas, todas open source, todas dockerizables:

PostgreSQL	Data Warehouse	Una sola DB con schemas bronze / silver / gold.
dbt-core	Transformación	Los modelos SQL que definen cada capa. Tests built-in.
Dagster	Orquestación + Gobierno	Corre los jobs, muestra lineage, freshness y la UI de admins.
Metabase	BI	UI no-técnica para que el usuario final haga sus dashboards.

¿Por qué este stack?

Las alternativas y qué descartamos



No es el único stack, pero es el más barato de aprender.



Airflow vs Dagster

Airflow tiene una curva de entrada más simple.



DuckDB vs Postgres

Postgres maneja mejor múltiples usuarios y herramientas simultáneas.



Superset vs Metabase

Metabase requiere menos configuración inicial.



dbt

Estándar de facto. Tests, documentación y lineage integrados.



Python + SQL

Evitamos agregar nuevos lenguajes.



Resultado: simple de aprender, barato de operar y listo para escalar.

Arquitectura

Cómo se conectan las piezas



Dagster dispara y monitorea todo el pipeline (lineage + freshness).

Todo corre en un único docker-compose: 5 servicios, una sola red.

Referencias

Para seguir leyendo

- Kimball, "The Data Warehouse Toolkit" — la biblia del star schema.
- Databricks blog → "What is a medallion architecture?"
- dbt docs → <https://docs.getdbt.com>
- Dagster docs → <https://docs.dagster.io>
- Metabase docs → <https://www.metabase.com/docs>